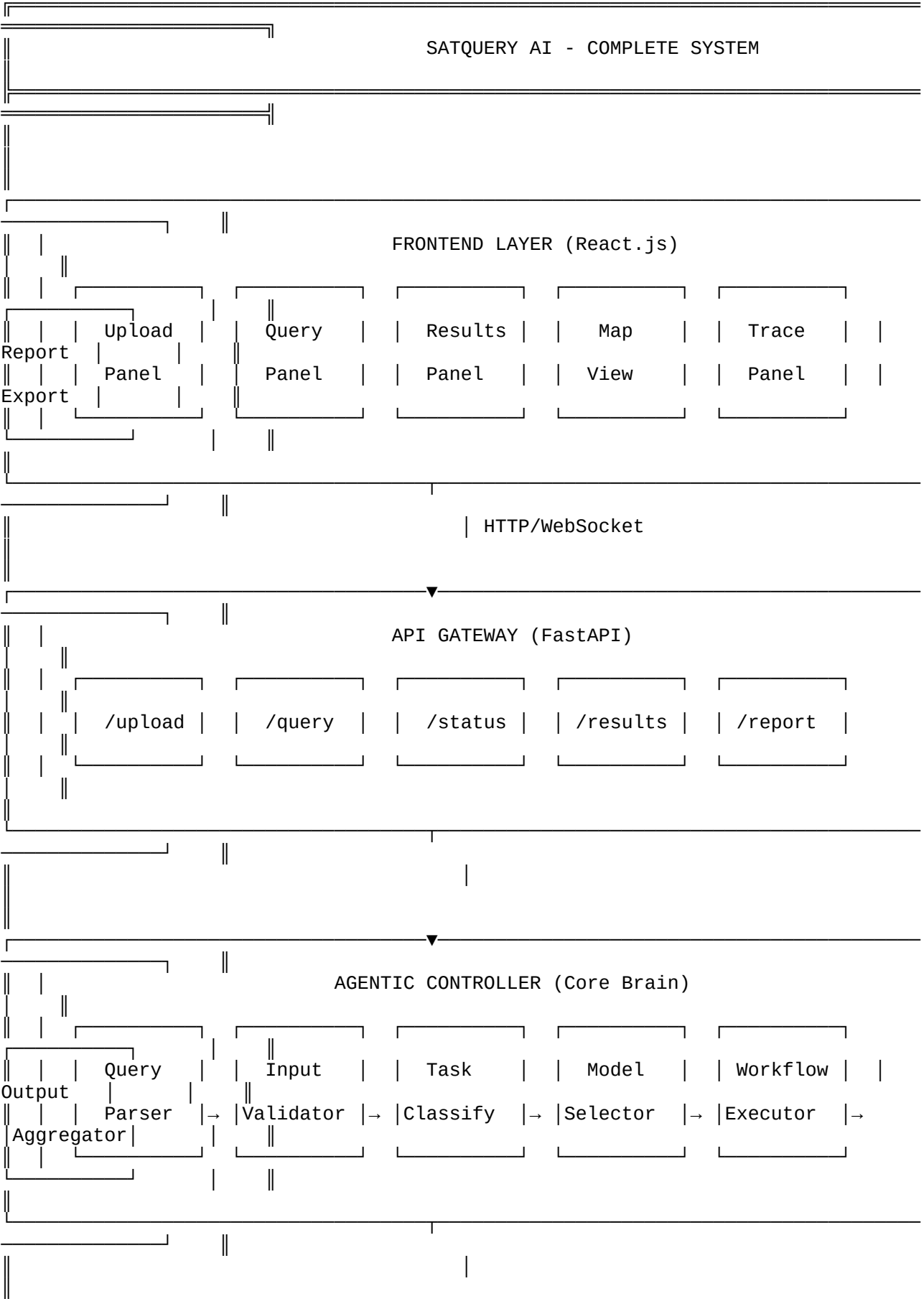
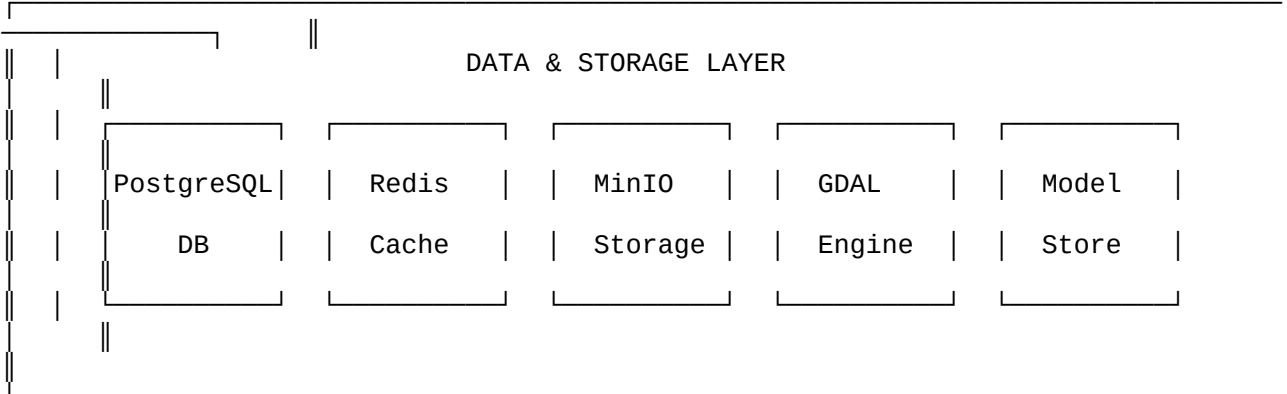
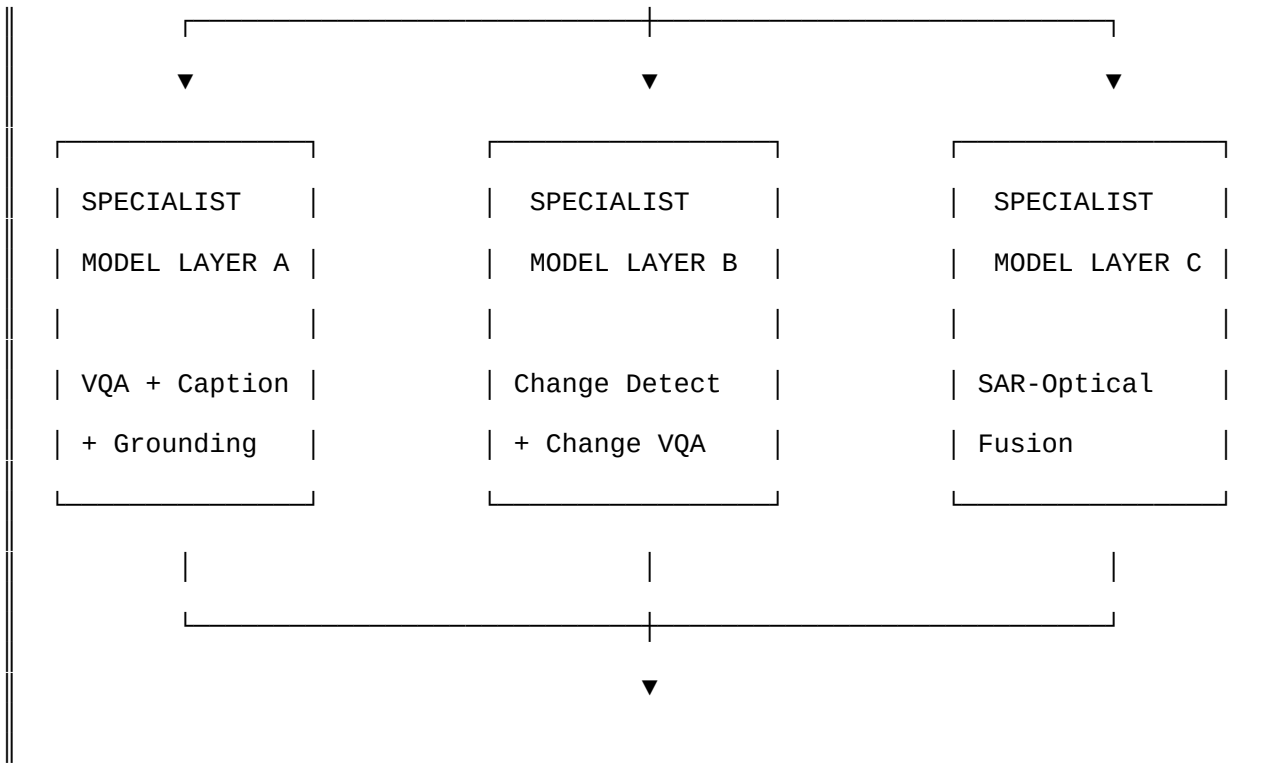


SatQuery AI - Complete Structural & Diagrammatic Representation

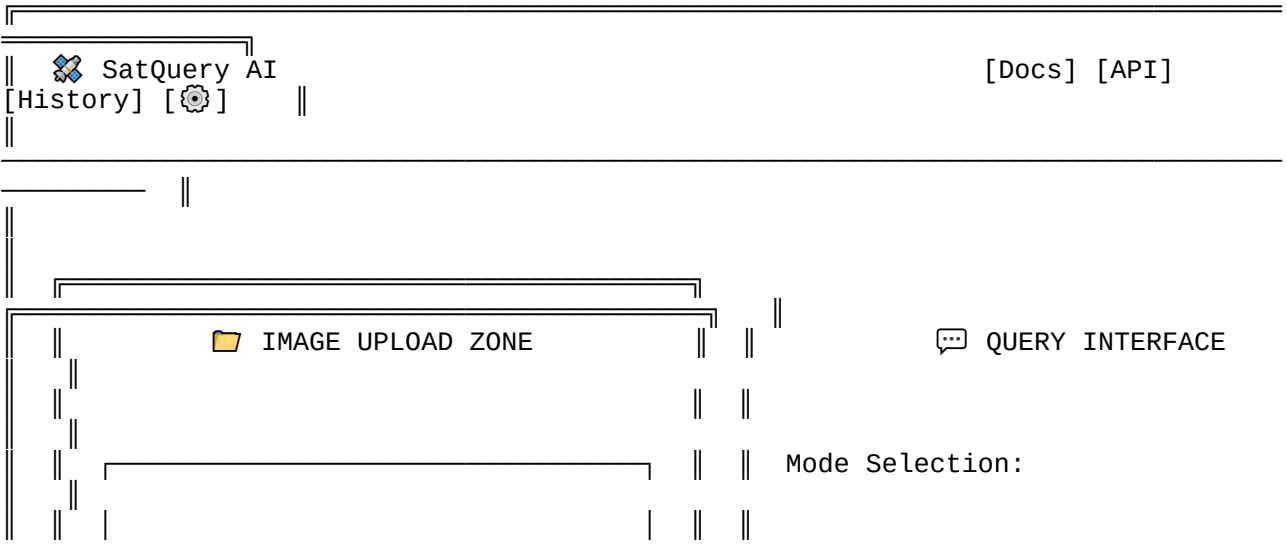
🏠 MASTER SYSTEM ARCHITECTURE

text





FRONTEND - COMPLETE VIEW
Screen 1: Main Landing / Upload Screen
text



SAR+

Optical

Drop Image(s) Here

Supports: GeoTIFF, TIFF

PNG, JPEG (benchmarks only)

image(s)...

[Browse Files] [URL Input]

IMAGE 1 STATUS:

✓ Format: GeoTIFF

✓ Modality: Optical/MS

✓ Bands: 13 (Sentinel-2)

✓ Resolution: 10m

✓ CRS: EPSG:32643

i

Size: 512x512 px

here"

dates?"

regions"

Templates

IMAGE 2 STATUS (Optional):

☐ Not uploaded yet

[+ Add Second Image]

Compatibility Check:

✓ Single Image Mode Ready

Single

Pair

Image

Temporal

Ask anything about your

Quick Query Suggestions:

"Describe the land cover

"Find all water bodies"

"What changed between

"Identify built-up

Voice Query

ANALYZE NOW

Screen 2: Processing / Results Screen
text

Query: "What changed between these two images and where did it occur?"

Mode: Bi-temporal Pair | Status: Complete | Time: 4.2s

Visual Evidence Panel

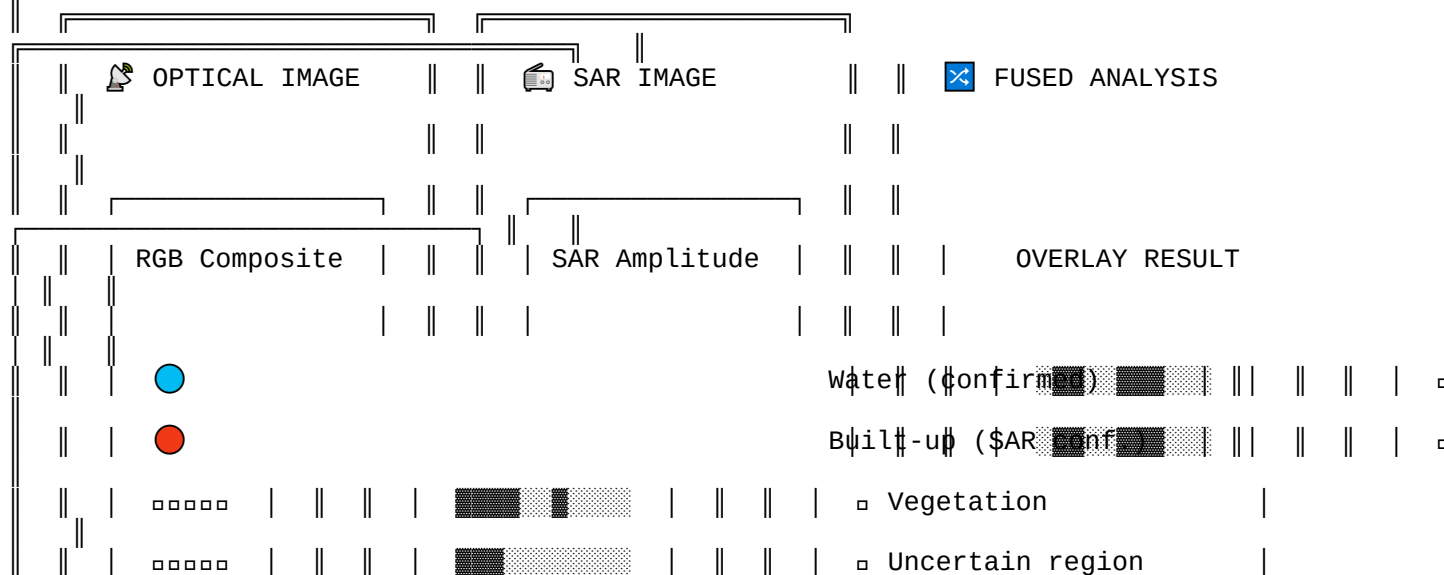
[T1: Before] ↔ [T2: After]

Change Map:

</

[▼]

[\[📄 Download Full Report PDF\]](#)
[\[🔄 Ask Follow-up\]](#)
[\[📊 View Metrics\]](#)

 SatQuery AI - Cross Modal Analysis

WATER BODY 1

Conf: 94%

Area: ~2.3 sq km

Type: Natural Lake

WATER BODY 2

Conf: 87%

BBox: [342,156,478,312]

Confidence: 87%

Area: ~1.1 sq km

Type: River/Stream

Region 2: Water Body (River)

Toggle Boxes

Toggle Masks

Download Annotated Image

Total Water: ~3.4 sq km (8.2% of AOI)

⚙️ BACKEND - COMPLETE ARCHITECTURE

Backend Layer Structure

text

BACKEND COMPLETE ARCHITECTURE

REQUEST FLOW:

Client → [Nginx] → [FastAPI Gateway] → [Celery Queue] → [Worker] → [Models] → [Response]

FASTAPI APPLICATION

/api/v1/

upload/

single

pair

validate

POST → ImageUploadRouter

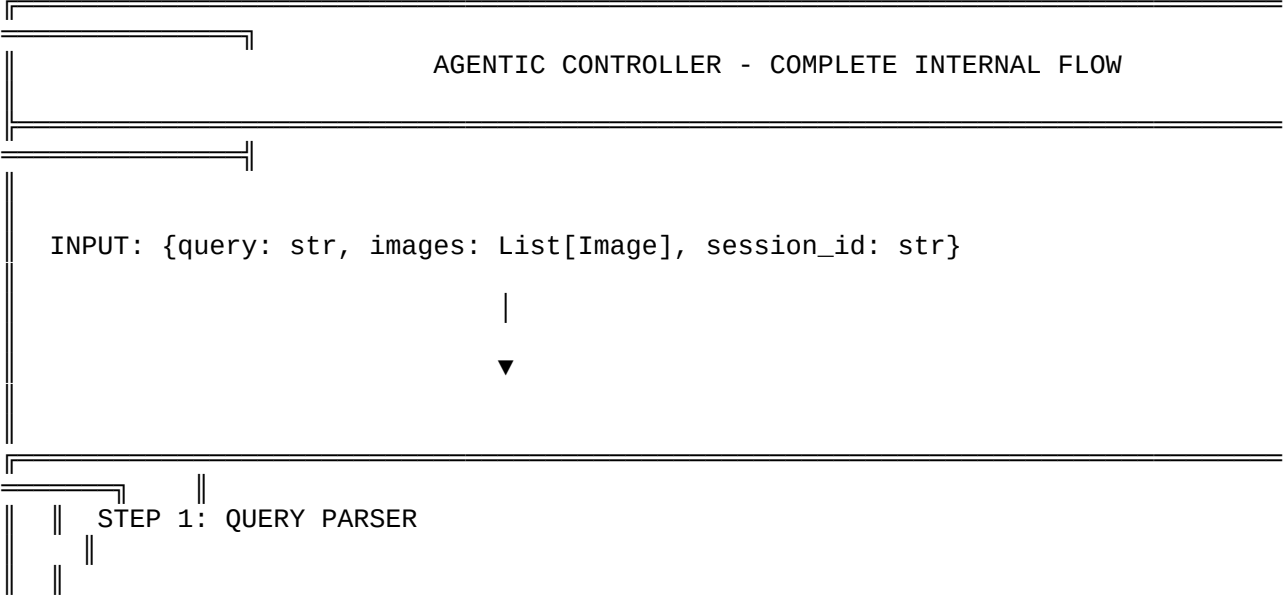
→ validate + store single image

→ validate + store image pair

→ return metadata + compatibility



 AGENTIC CONTROLLER - INTERNAL FLOW
text



Input Query → LLM Intent Extractor (Mistral-7B)

Extracts:

```
intent_keywords: ["changed", "between", "where"]
spatial_reference: True
temporal_reference: True
target_object: ["changes", "area"]
task_hint: "CHANGE_DETECTION"
```



STEP 2: INPUT VALIDATOR

For each image run:

FORMAT CHECK	MODALITY DETECTION	METADATA EXTRACTION
- File extension - Magic bytes check - GDAL readable? - Corrupt check	- Band count check - SAR metadata check - Wavelength bands - Polarization info	- CRS extraction - Bounding box - Acquisition date - Sensor type

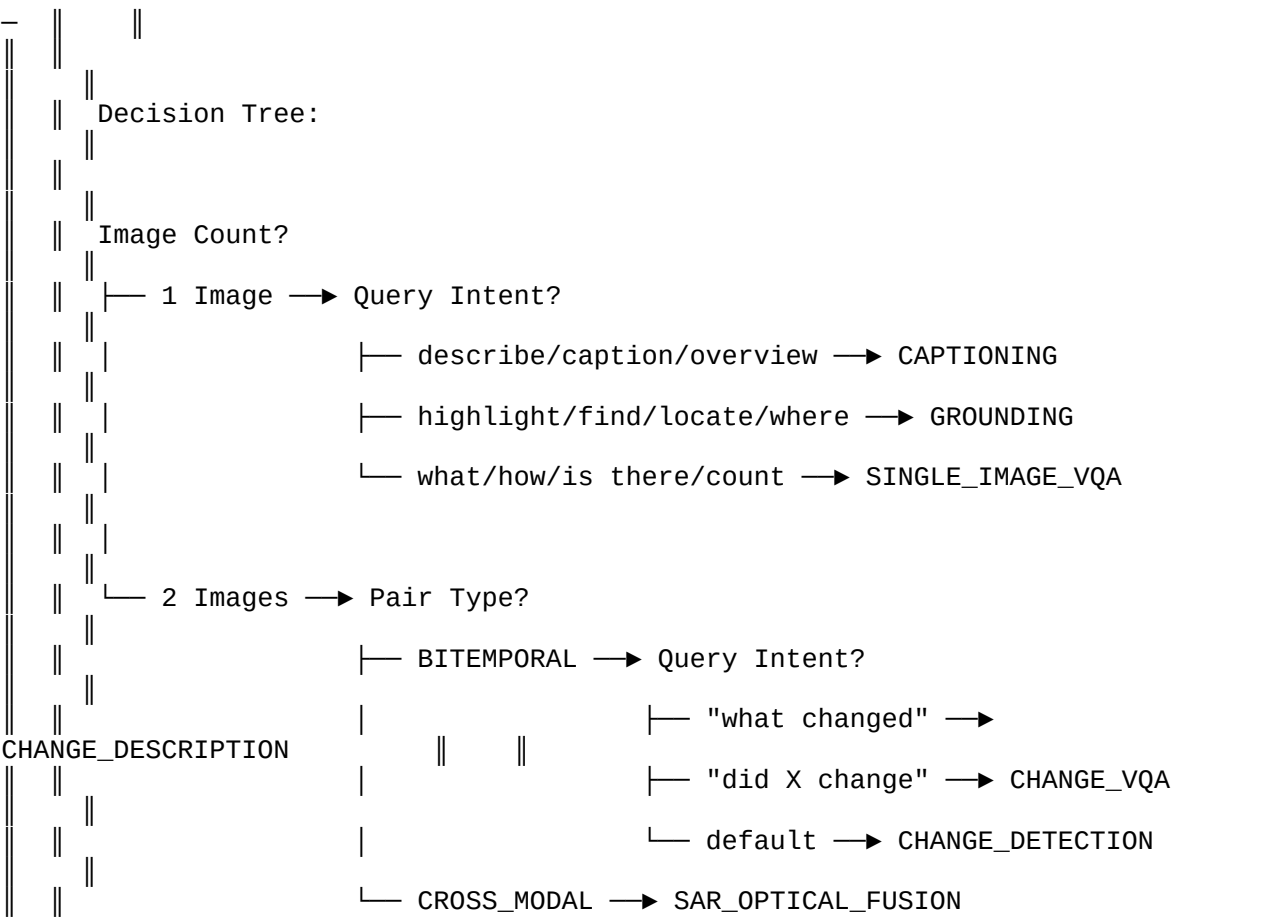
For image pairs additionally run:

CHECK			COREGISTRATION CHECK	PAIR TYPE DETECTION	COMPATIBILITY
			- BBox overlap calc - CRS match - Resolution match	- Same date = error - Diff dates = temporal - Diff modality = cross	- Same - Same area - Band

OUTPUT: ValidationResult {valid, config, warnings, metadata}



STEP 3: TASK CLASSIFIER



OUTPUT: TaskType + SubTaskType + Priority

|



STEP 4: MODEL SELECTOR (Registry Lookup)

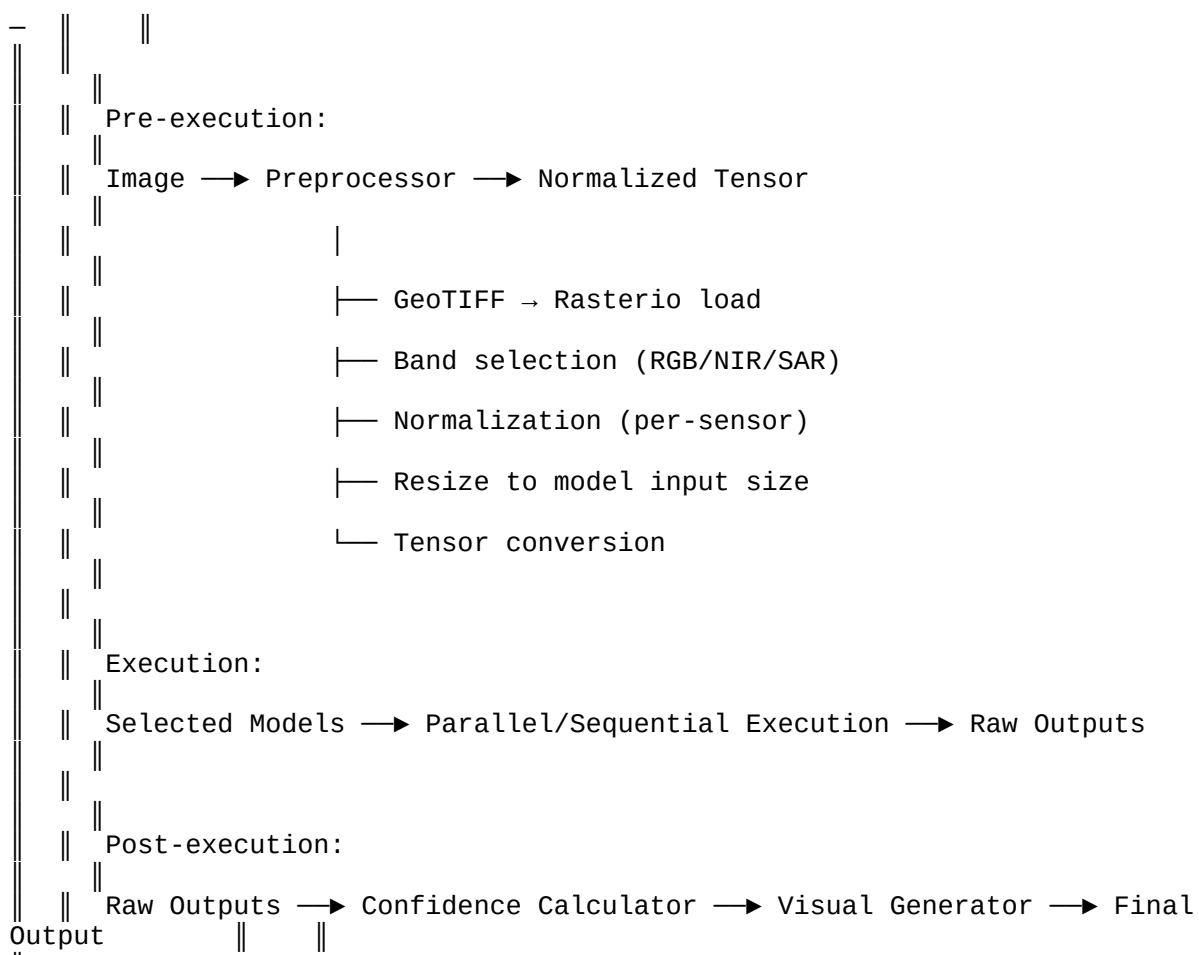
MODEL REGISTRY:

Task	Primary Model	Fallback Model
CAPTIONING	GeoChat-7B	BLIP-2 + RS-CLIP
SINGLE_IMAGE_VQA	RS-LLaVA (fine-tuned)	GeoChat-7B
GROUNDING	Grounding-DINO-RS	GLIP + SAM
CHANGE_DETECTION	ChangeFormer-v2	SiamSeg
CHANGE_VQA	CDVQA-Decoder	ChangeFormer + LLM
CHANGE_DESCRIPTION	ChangeFormer + GeoChat	BIT + BLIP-2
SAR_OPTICAL_FUSION	Dual-Encoder-Fusion	Concatenation + LLM
RS_EMBEDDING	RemoteCLIP (always active)	BigEarthNet-CLIP

|



STEP 5: WORKFLOW EXECUTOR



STEP 6: OUTPUT AGGREGATOR

Combines:

- Text answers from VQA/Captioning models
- Change maps from ChangeFormer
- Bounding boxes from Grounding model
- SAR analysis from Fusion model
- Confidence scores across all models
- Execution trace log

```
||
|| OUTPUT: AgentResponse {text, visual_evidence, confidence, trace, metadata}
||
```



SPECIALIST MODEL PIPELINES

Pipeline A: Single Image VQA + Captioning
text

PIPELINE A: SINGLE IMAGE VQA + CAPTIONING

INPUT IMAGE (GeoTIFF/PNG)

|



IMAGE PREPROCESSOR

| GDAL/Rasterio |

| Band Select |

| Normalize |

| Load File |

| (RGB or NIR) |

| (0-1 range) |

|



| Metadata |

| Extraction |

| (CRS, Date) |

|



| Resize to |

| 224x224 or |

| 336x336 |

|



| RemoteCLIP Encoder |

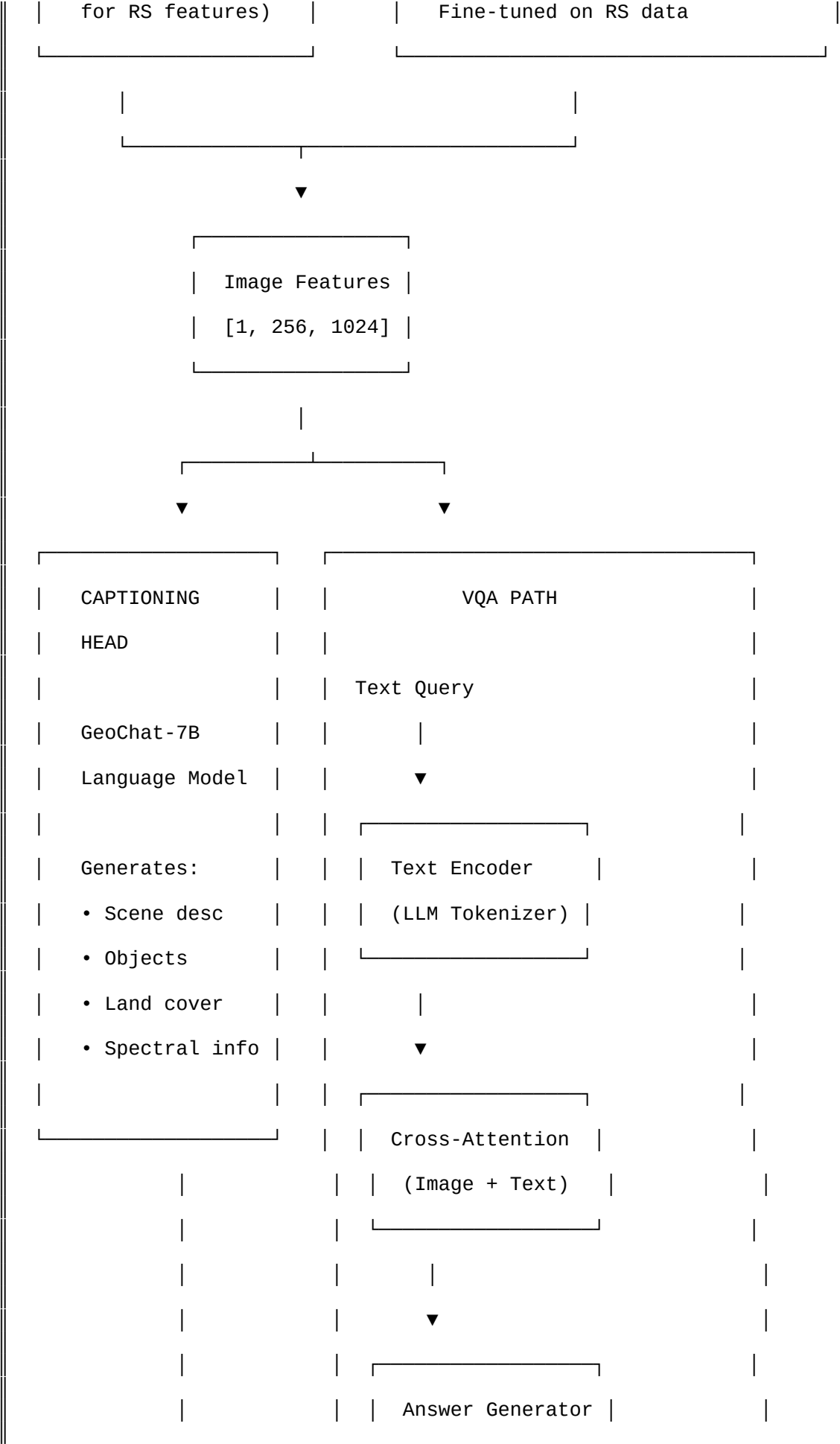
| (BigEarthNet adapted) |

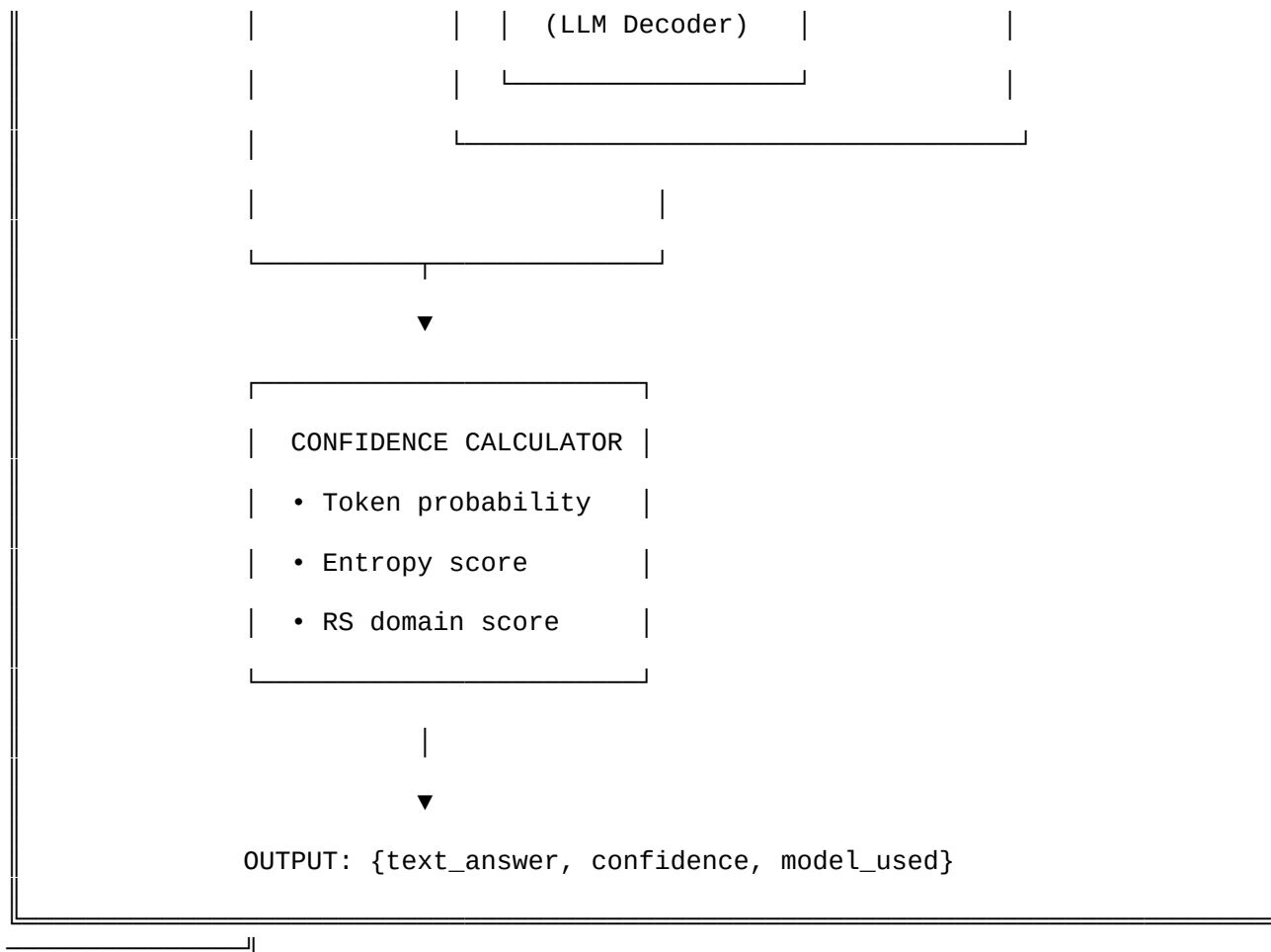
|



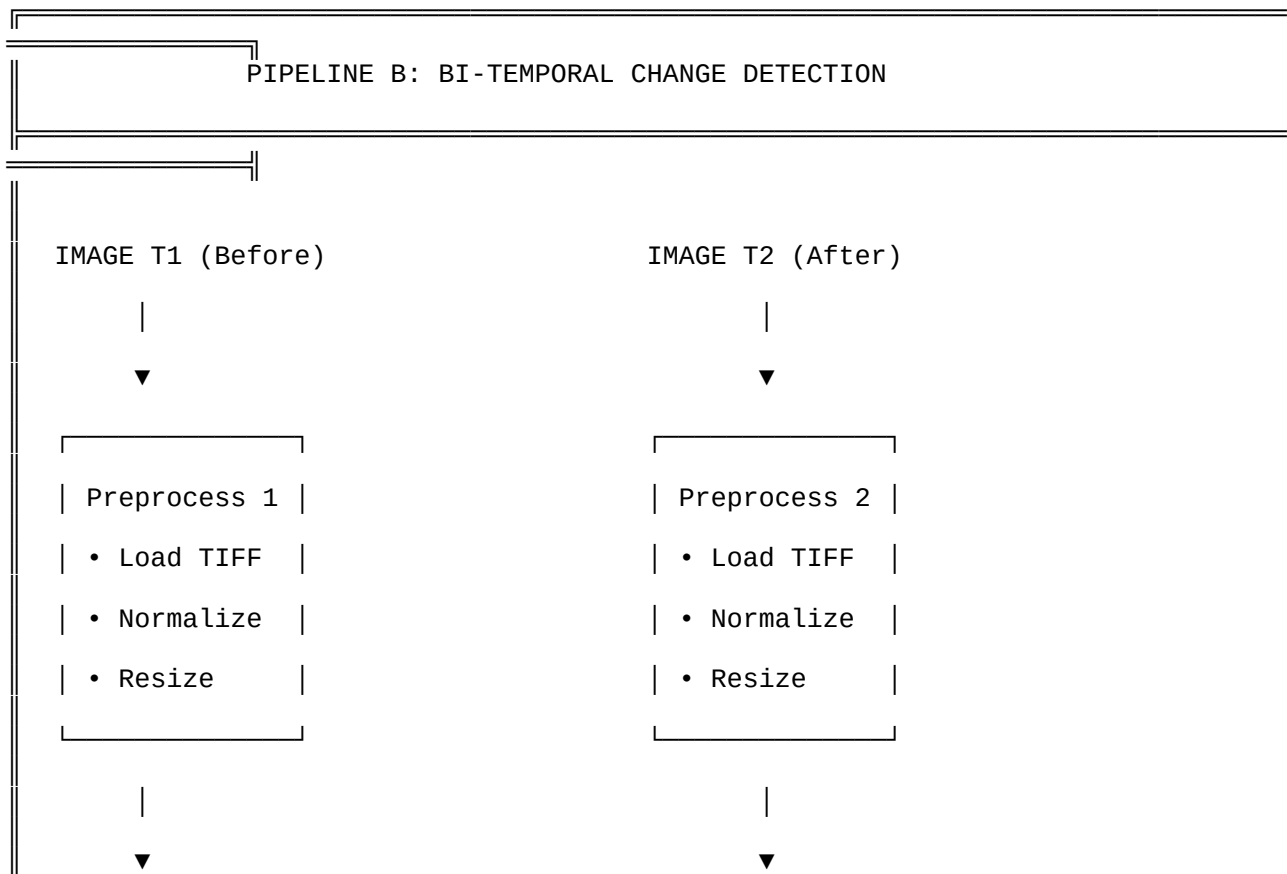
| Visual Encoder |

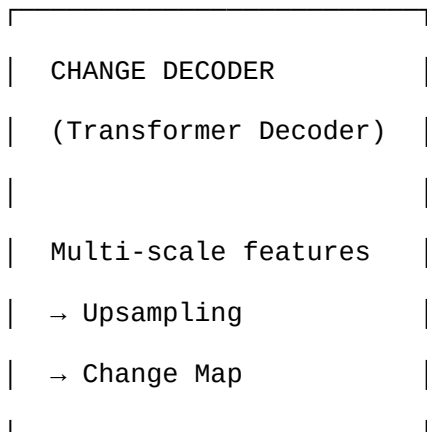
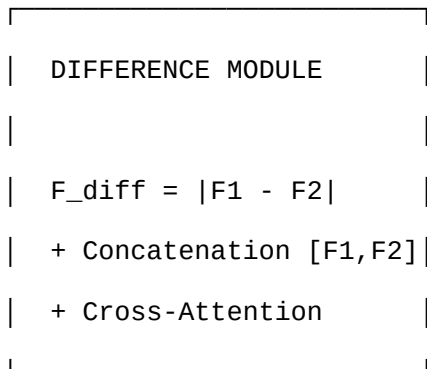
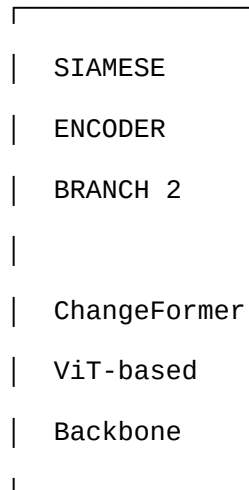
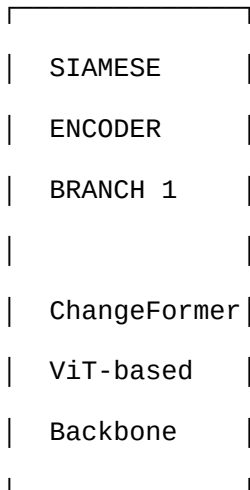
| (ViT-L/14 or CLIP ViT-B/32) |

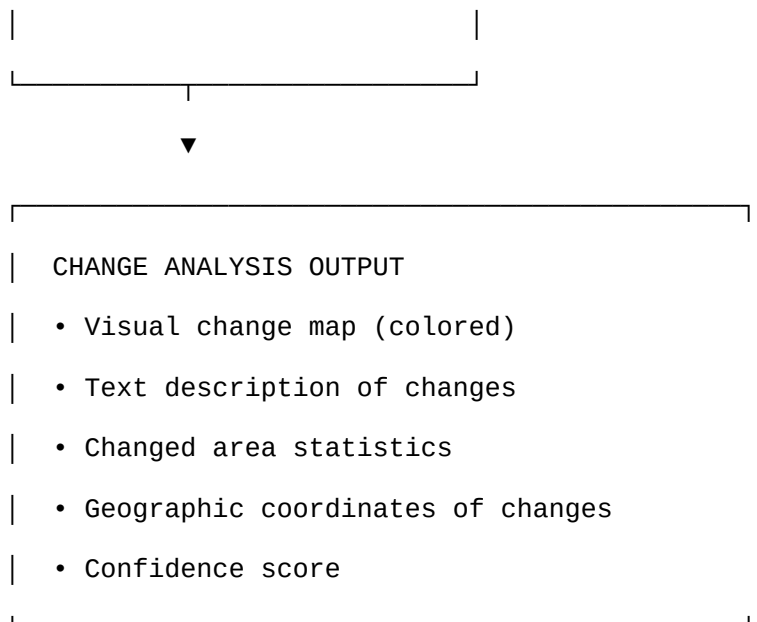
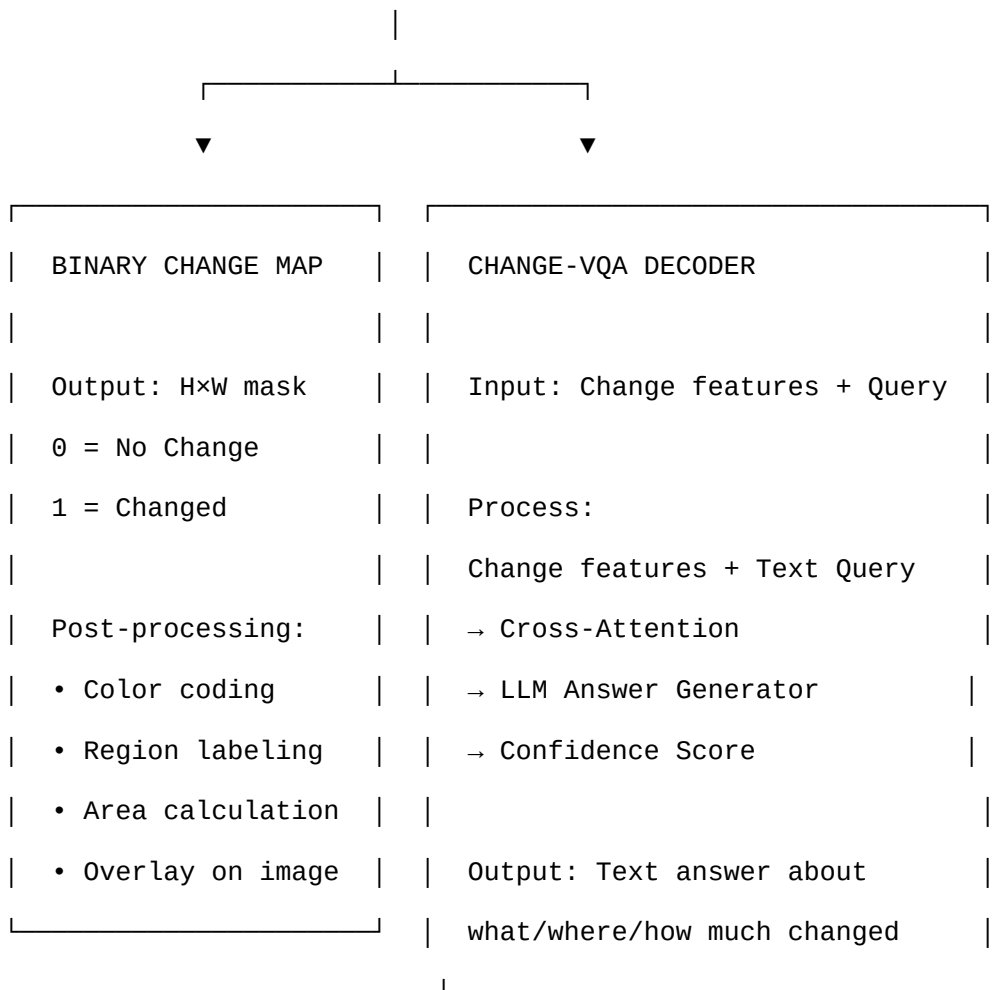




Pipeline B: Change Detection
text







Pipeline C: SAR + Optical Fusion
text

PIPELINE C: SAR-OPTICAL CROSS-MODAL FUSION

OPTICAL IMAGE

(Sentinel-2/Cartosat-2S)

Bands: B2,B3,B4,B8 (RGB+NIR)

|



OPTICAL BRANCH

Preprocessing:

- Atmospheric corr
- Cloud masking
- Radiometric cal
- Band stacking

|



OPTICAL ENCODER

RemoteCLIP/ViT

Fine-tuned on

BigEarthNet S2

Output:

[B, 512, 14, 14]

Optical Features

|

|

SAR IMAGE

(Sentinel-1/RISAT)

Bands: VV, VH (Polarization)

|



SAR BRANCH

Preprocessing:

- dB conversion
- Speckle filtering
- Geocoding
- Lee filter

|



SAR ENCODER

SAR-specific ViT

- Adapted for SAR backscatter patterns

- BigEarthNet S1

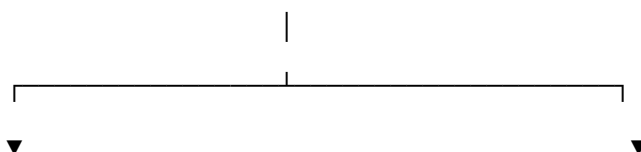
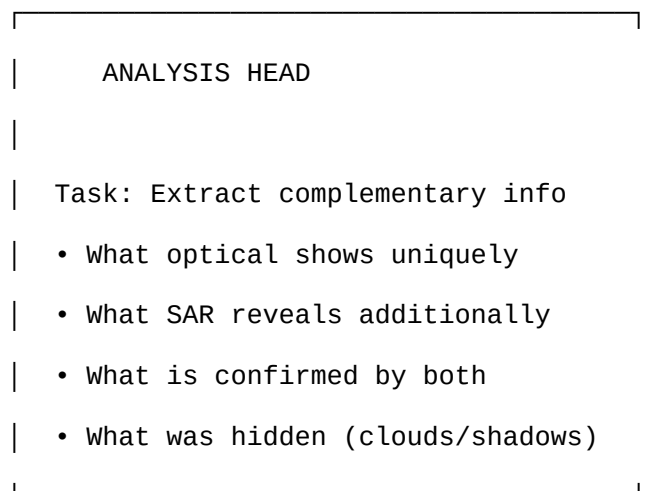
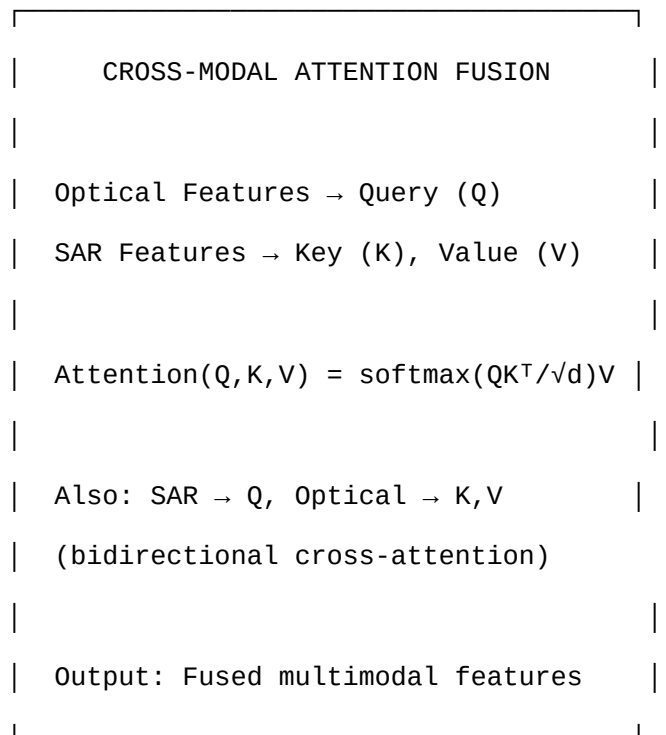
fine-tuned

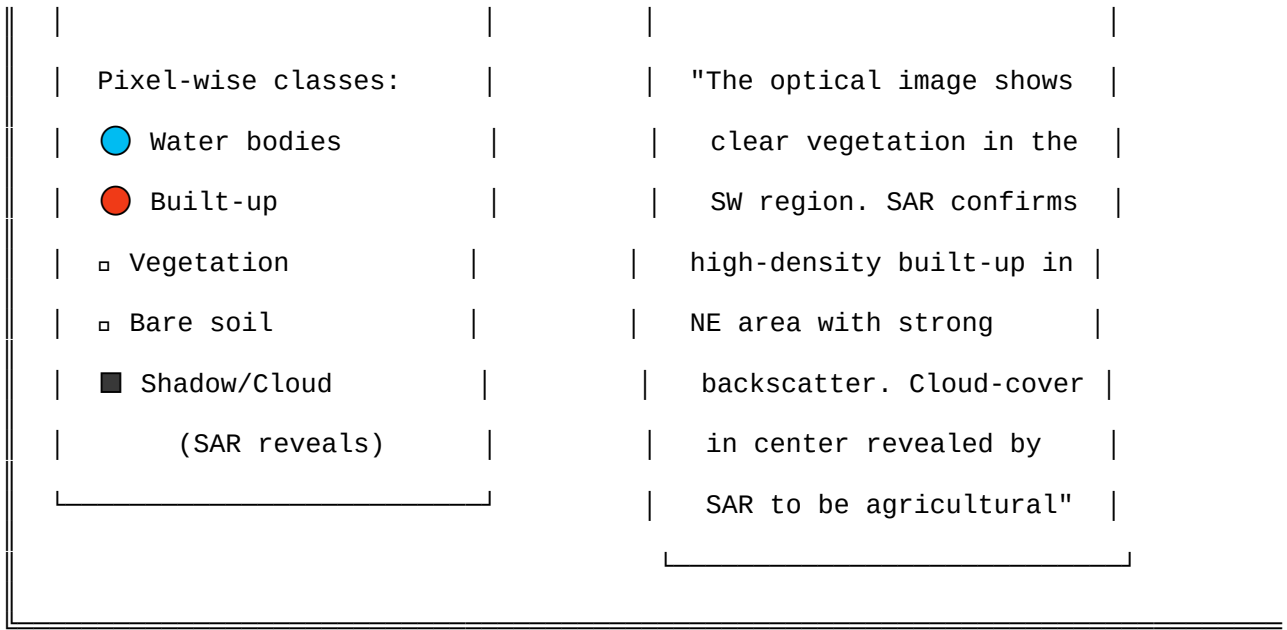
Output:

[B, 512, 14, 14]

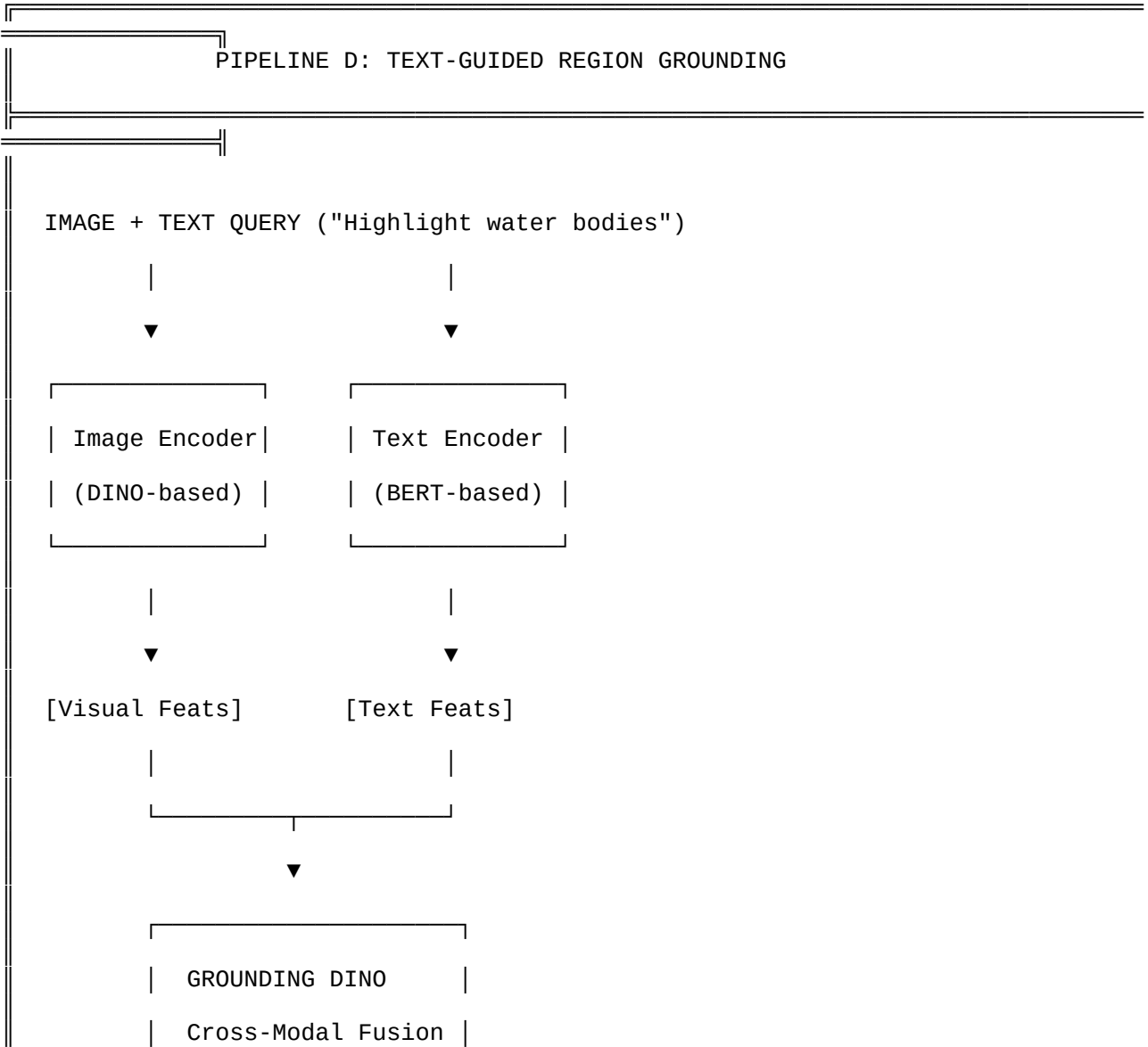
SAR Features

|





Pipeline D: Text-Guided Grounding
text



| + Phrase Grounding |

|



| DETECTION HEAD |

| Bounding Boxes |

| + Confidence scores |

|



| SAM SEGMENTATION |

| (Pixel-level masks) |

| from bbox prompts |

|



| OUTPUT: Annotated Image |

| • Bounding boxes drawn |

| • Pixel masks overlaid |

| • Region labels |

| • Area calculations |



DATA FLOW ARCHITECTURE

text

COMPLETE DATA FLOW

USER BROWSER

1. User uploads GeoTIFF files
2. User types natural language query
3. Clicks "Analyze"

|
| multipart/form-data POST



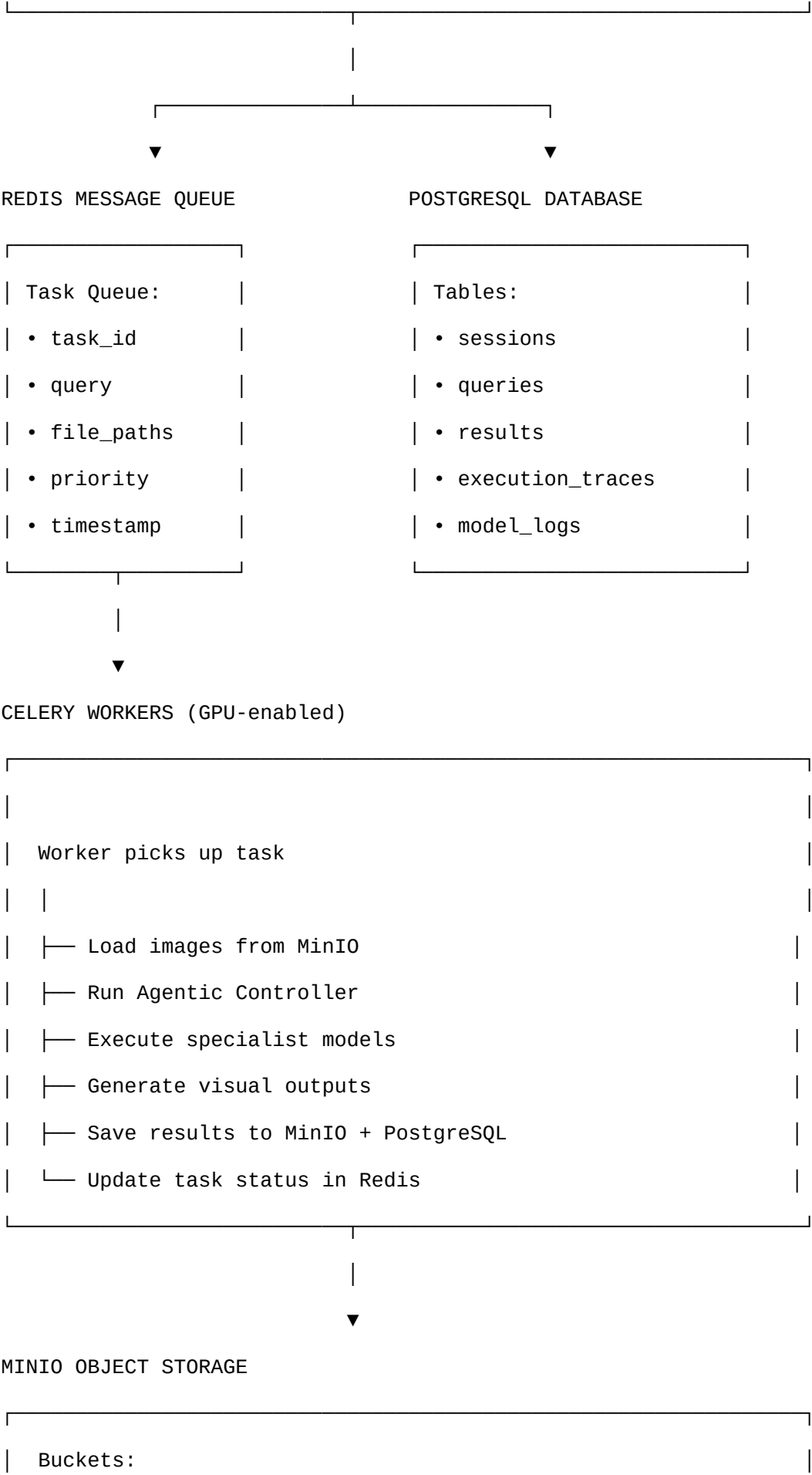
NGINX (Reverse Proxy + Load Balancer)

- SSL termination
- Static file serving
- Rate limiting
- Request routing



FASTAPI APPLICATION SERVER

- POST /api/v1/upload
 - └─ Receive files
 - └─ Save to MinIO storage
 - └─ Extract metadata via GDAL
 - └─ Return: {file_ids, metadata, compatibility}
- POST /api/v1/query
 - └─ Receive {query, file_ids}
 - └─ Create Celery task
 - └─ Store in Redis queue
 - └─ Return: {task_id, status: "queued"}



```
| |─ /uploads/{session_id}/{file_id}.tif |
| |─ /results/{task_id}/change_map.png |
| |─ /results/{task_id}/annotated_image.png |
| |─ /reports/{task_id}/report.pdf |
|_|
```

```
|
| WebSocket push / polling
```



CLIENT RECEIVES RESULT

```
|_|
| GET /api/v1/query/result/{task_id} |
| Returns: |
| { |
|   text_answer: "...", |
|   confidence: 0.87, |
|   change_map_url: "...", |
|   annotated_image_url: "...", |
|   execution_trace: [...], |
|   report_url: "..." |
| } |
|_|
```

MODEL TRAINING & FINE-TUNING PIPELINE

text

BIGEARTHNET FINE-TUNING PIPELINE (RS ADAPTATION)

BIGEARTHNET DATASET

```
|_|
| 590,326 patches |
| |─ Sentinel-2 (S2): 12 spectral bands, 10-60m resolution |
| |─ Sentinel-1 (S1): VV+VH polarizations, 10m resolution |
|_|
```

| Labels: 43 land-use/cover classes |

|



| DATA PREPARATION |

| S2 Images |

| |— Select bands: B2(Blue),B3(Green),B4(Red),B8(NIR) |

| |— Normalize: (pixel - mean) / std per band |

| |— Resize: 128×128 → 224×224 |

| |— Augment: flip, rotate, color jitter |

| S1 Images |

| |— Convert to dB scale: $10 \cdot \log_{10}(\text{amplitude}^2)$ |

| |— Clip range: [-25dB, 0dB] |

| |— Stack VV + VH as 2-channel image |

| |— Resize: 128×128 → 224×224 |

| Text Labels |

| |— "Continuous urban fabric" → "Dense urban residential area" |

| |— "Arable land" → "Open farmland used for crop cultivation" |

| |— Multi-label → Concatenated description |



| OPTICAL CLIP | | SAR ENCODER TRAINING |

| FINE-TUNING | | |

| | | Base: ViT-B/32 |

| Base: CLIP | | Modified: 2-channel input layer |

Method:	Training: S1 patches + labels
Contrastive	Loss: Cross-entropy classification
Loss	Output: SAR feature extractor
Image-Text	
pairs from	
BigEarthNet	



TRAINING CONFIGURATION	
Optimizer:	AdamW (lr=1e-5, weight_decay=0.01)
Scheduler:	Cosine annealing with warmup
Batch size:	128 (gradient accumulation: 4)
Epochs:	20 (early stopping on val loss)
Hardware:	4× A100 GPUs (or Colab A100)
Mixed Prec:	FP16 (AMP)
Data Split:	80% train / 10% val / 10% test



EVALUATION METRICS	
RSVQA:	Accuracy on presence/comparison/count questions
VRSBench:	CIDEr, BLEU-4, METEOR for captioning
	mAP for grounding
CDVQA:	Accuracy on change-based questions
BigEarthNet:	mAP on multi-label classification

📁 COMPLETE PROJECT FOLDER STRUCTURE
text

PROJECT DIRECTORY STRUCTURE

satquery-ai/

```
|
|
├─ frontend/                                ← React.js Application
|   └─ public/
|       └─ index.html
|       └─ assets/
|       |
|       └─ src/
|           └─ components/
|               └─ Layout/
|                   └─ Navbar.jsx
|                   └─ Sidebar.jsx
|                   └─ Upload/
|                       └─ ImageUploadZone.jsx    ← Drag & drop
|                       └─ ImagePreview.jsx        ← Thumbnail + metadata
|                       └─ CompatibilityChecker.jsx ← Real-time validation
|                       └─ MetadataDisplay.jsx     ← CRS, bands, date
|                   └─ Query/
|                       └─ QueryInput.jsx          ← Text input + voice
|                       └─ QuerySuggestions.jsx     ← Example queries
|                       └─ ModeSelector.jsx        ← Single/Pair/SAR mode
|                   └─ Results/
|                       └─ ResultsPanel.jsx        ← Main results container
|                       └─ TextAnswer.jsx          ← AI text response
|                       └─ ConfidenceMeter.jsx     ← Confidence visualization
```

				└─ VisualEvidence.jsx	← Image with overlays
				└─ StatisticsPanel.jsx	← Area, counts, metrics
				└─ Map/	
				└─ MapView.jsx	← Leaflet map container
				└─ ChangeMapOverlay.jsx	← Change map layer
				└─ BoundingBoxOverlay.jsx	← Grounding boxes
				└─ ImageCompare.jsx	← Before/After slider
				└─ Trace/	
				└─ ExecutionTrace.jsx	← Agent steps log
				└─ TraceStep.jsx	← Individual step card
				└─ ModelInfo.jsx	← Model used details
				└─ Export/	
				└─ ReportDownload.jsx	← PDF download button
				└─ JsonExport.jsx	← Raw JSON export
				└─ pages/	
				└─ HomePage.jsx	← Main interface
				└─ HistoryPage.jsx	← Past queries
				└─ DocsPage.jsx	← Documentation
				└─ AboutPage.jsx	← About SatQuery AI
				└─ hooks/	
				└─ useImageUpload.js	← Upload logic
				└─ useQuerySubmit.js	← Query submission
				└─ usePolling.js	← Status polling
				└─ useWebSocket.js	← Real-time updates
				└─ services/	
				└─ api.js	← Axios API client
				└─ imageService.js	← Image handling
				└─ reportService.js	← Report generation

			└─ store/	← Redux/Zustand state
			└─ uploadSlice.js	
			└─ querySlice.js	
			└─ resultsSlice.js	
			└─ utils/	
			└─ imageUtils.js	← Client-side image ops
			└─ colorMaps.js	← Visualization colors
			└─ backend/	← FastAPI Application
			└─ app/	
			└─ main.py	← FastAPI entry point
			└─ config.py	← Settings & env vars
			└─ api/	
			└─ routes/	
			└─ upload.py	← Upload endpoints
			└─ query.py	← Query endpoints
			└─ results.py	← Results endpoints
			└─ export.py	← Export endpoints
			└─ middleware/	
			└─ auth.py	← Authentication
			└─ cors.py	← CORS settings
			└─ agent/	← AGENTIC CONTROLLER
			└─ controller.py	← Main agent orchestrator
			└─ query_parser.py	← NLP query understanding
			└─ input_validator.py	← Image validation
			└─ task_classifier.py	← Task routing logic
			└─ model_selector.py	← Model registry lookup
			└─ workflow_executor.py	← Model execution
			└─ output_aggregator.py	← Result combination

			└─ model_registry.py	← Available models dict
			└─ models/	← SPECIALIST ML MODELS
			└─ base_model.py	← Abstract model class
			└─ rs_vqa/	
			└─ model.py	← RS-LLaVA VQA model
			└─ preprocessor.py	← Input preprocessing
			└─ postprocessor.py	← Output formatting
			└─ captioning/	
			└─ model.py	← GeoChat captioning
			└─ preprocessor.py	
			└─ grounding/	
			└─ model.py	← Grounding-DINO + SAM
			└─ visualizer.py	← BBox drawing
			└─ change_detection/	
			└─ model.py	← ChangeFormer
			└─ change_vqa.py	← Change VQA decoder
			└─ visualizer.py	← Change map coloring
			└─ sar_optical_fusion/	
			└─ model.py	← Dual encoder fusion
			└─ sar_preprocessor.py	← SAR-specific preprocessing
			└─ fusion_head.py	← Cross-attention fusion
			└─ rs_encoder/	
			└─ remote_clip.py	← RemoteCLIP wrapper
			└─ bigearthnet_clip.py	← Fine-tuned CLIP
			└─ processing/	← IMAGE PROCESSING
			└─ geotiff_handler.py	← GDAL/Rasterio operations
			└─ sar_preprocessor.py	← SAR-specific processing
			└─ image_normalizer.py	← Normalization logic
			└─ coregistration.py	← Pair alignment checking
			└─ metadata_extractor.py	← Geospatial metadata

```

|   |   |
|   |   └─ tasks/                ← CELERY TASKS
|   |       └─ celery_app.py      ← Celery configuration
|   |       └─ analysis_task.py   ← Main analysis task
|   |       └─ report_task.py     ← PDF generation task
|   |   |
|   |   └─ services/             ← BUSINESS LOGIC
|   |       └─ storage_service.py  ← MinIO operations
|   |       └─ cache_service.py    ← Redis operations
|   |       └─ db_service.py       ← PostgreSQL operations
|   |       └─ report_service.py   ← PDF report generation
|   |       └─ confidence_service.py ← Confidence calculation
|   |   |
|   |   └─ utils/
|   |       └─ logger.py           ← Logging configuration
|   |       └─ validators.py       ← Input validation helpers
|   |       └─ color_maps.py       ← Visualization utilities
|   |
|   └─ tests/
|       └─ test_agent.py
|       └─ test_models.py
|       └─ test_api.py
|       └─ test_processing.py
|
└─ training/                      ← ML TRAINING SCRIPTS
    └─ bigearthnet/
        └─ download_data.py        ← Dataset download
        └─ preprocess.py           ← Data preparation
        └─ train_clip.py           ← CLIP fine-tuning
        └─ train_sar_encoder.py    ← SAR encoder training
        └─ evaluate.py             ← Evaluation script
    |

```

			└─ vqa/	
				└─ train_rsvqa.py
				└─ VQA fine-tuning on RSVQA
				└─ evaluate_rsvqa.py
				└─ VQA evaluation
				└─ change_detection/
				└─ train_changeformer.py
				└─ Change detection training
				└─ evaluate_cdvqa.py
				└─ CDVQA evaluation
				└─ models/
				└─ SAVED MODEL WEIGHTS
				└─ remote_clip/
				└─ RemoteCLIP weights
				└─ bigearthnet_clip/
				└─ Fine-tuned weights
				└─ rs_llava/
				└─ VQA model weights
				└─ changeformer/
				└─ Change detection weights
				└─ fusion/
				└─ SAR-Optical fusion weights
				└─ docker/
				└─ DEPLOYMENT
				└─ Dockerfile.frontend
				└─ Dockerfile.backend
				└─ Dockerfile.worker
				└─ docker-compose.yml
				└─ Full stack setup
				└─ configs/
				└─ CONFIGURATION FILES
				└─ model_registry.yaml
				└─ Model definitions
				└─ task_routing.yaml
				└─ Routing rules
				└─ training_config.yaml
				└─ Training hyperparameters
				└─ docs/
				└─ DOCUMENTATION
				└─ api_reference.md
				└─ model_cards/
				└─ demo_guide.md

REST API COMPLETE SCHEMA

POST /api/v1/upload

Request: multipart/form-data

files: [image1.tif, image2.tif]

session_id: "uuid"

Response: {

"file_ids": ["fid_001", "fid_002"],

"images": [

{

"file_id": "fid_001",

"format": "GeoTIFF",

"modality": "optical",

"bands": 13,

"resolution": "10m",

"crs": "EPSG:32643",

"bbox": [72.5, 23.1, 73.2, 23.8],

"acquisition_date": "2024-03-15",

"sensor": "Sentinel-2"

}

],

"compatibility": {

"pair_type": "BITEMPORAL",

"coregistered": true,

"warnings": []

}

}

POST /api/v1/query

Request: {
 "query": "What changed between these two images?",
 "file_ids": ["fid_001", "fid_002"],
 "session_id": "uuid"
}

Response: {
 "task_id": "task_abc123",
 "status": "queued",
 "estimated_time": 45
}

GET /api/v1/query/result/{task_id}

Response: {
 "task_id": "task_abc123",
 "status": "completed",
 "result": {
 "text_answer": "Significant urban expansion detected...",
 "confidence": 0.87,
 "task_type": "CHANGE_DETECTION",
 "visual_evidence": {
 "change_map_url": "/results/task_abc123/change_map.png",
 "annotated_image_url": "/results/task_abc123/annotated.png",
 "bounding_boxes": [{"label": "urban", "bbox": [x,y,w,h], "conf": 0.9}]
 },
 "statistics": {
 "changed_area_km2": 12.4,
 "change_percentage": 23.1,
 "change_types": {"urban_growth": 8.2, "vegetation_loss": 4.2} }
 }
}

```

    },
    "execution_trace": [
        {
            "step": 1, "action": "query_parsed", "result": "CHANGE_DETECTION",
            "time": 0.3},
        {
            "step": 2, "action": "input_validated", "result": "BITEMPORAL_PAIR",
            "time": 0.1},
        {
            "step": 3, "action": "model_selected", "result": "ChangeFormer-v2",
            "time": 0.05},
        {
            "step": 4, "action": "preprocessing", "result": "success", "time":
            1.2},
        {
            "step": 5, "action": "changeformer_run", "result": "map_generated",
            "time": 2.1},
        {
            "step": 6, "action": "vqa_decoder", "result": "answer_generated",
            "time": 0.8},
        {
            "step": 7, "action": "aggregation", "result": "complete", "time": 0.2}
    ],
    "model_info": {
        "primary_model": "ChangeFormer-v2",
        "language_model": "CDVQA-Decoder",
        "encoder": "RemoteCLIP (BigEarthNet-adapted)"
    },
    "report_url": "/reports/task_abc123/report.pdf"
}

```

 DOCKER DEPLOYMENT ARCHITECTURE
text

DOCKER COMPOSE ARCHITECTURE

docker-compose.yml

```

services:

```

```

    nginx:

```

← Port 80/443, serves frontend, proxies API

		image: nginx:alpine	
		ports: ["80:80"]	
		frontend:	← React build served by Nginx
		build: ./frontend	
		backend:	← FastAPI on port 8000
		build: ./backend	
		ports: ["8000:8000"]	
		environment:	
		- DATABASE_URL=postgresql://...	
		- REDIS_URL=redis://redis:6379	
		- MINIO_URL=minio:9000	
		worker:	← Celery GPU worker
		build: ./backend	
		command: celery -A app.tasks worker	
		deploy:	
		resources:	
		reservations:	
		devices:	
		- driver: nvidia	
		count: 1	



 DATABASE SCHEMA
text

POSTGRESQL DATABASE SCHEMA			
TABLE: sessions			
id	UUID	Primary Key	
created_at	TIMESTAMP	Session creation time	
last_active	TIMESTAMP	Last activity	
ip_address	VARCHAR(45)	Client IP	
TABLE: uploaded_images			

id	UUID	Primary Key
session_id	UUID	FK → sessions
file_path	TEXT	MinIO path
format	VARCHAR(20)	GeoTIFF/PNG/JPEG
modality	VARCHAR(20)	optical/sar/multispectral
bands	INTEGER	Number of bands
width	INTEGER	Image width
height	INTEGER	Image height
resolution	FLOAT	Ground resolution (meters)
crs	VARCHAR(50)	Coordinate reference system
bbox	JSONB	[min_lon, min_lat, max_lon, max_lat]
acquisition_dt	DATE	Image acquisition date
sensor	VARCHAR(50)	Sentinel-2/RISAT etc
metadata	JSONB	Full metadata dict
uploaded_at	TIMESTAMP	Upload timestamp

TABLE: analysis_tasks

id	UUID	Primary Key (task_id)
session_id	UUID	FK → sessions
query_text	TEXT	User's natural language query
image_ids	UUID[]	Array of FK → uploaded_images
task_type	VARCHAR(50)	CAPTIONING/VQA/GROUNDING/CHANGE/SAR_OPTICAL
status	VARCHAR(20)	queued/processing/completed/failed
created_at	TIMESTAMP	Task creation
completed_at	TIMESTAMP	Task completion
processing_sec	FLOAT	Time taken

TABLE: analysis_results

id	UUID	Primary Key
task_id	UUID	FK → analysis_tasks
text_answer	TEXT	AI-generated text response
confidence	FLOAT	Overall confidence score (0-1)
visual_outputs	JSONB	{change_map_url, annotated_url, bboxes}
statistics	JSONB	{area, percentages, counts}
models_used	JSONB	{primary, language, encoder}
report_url	TEXT	PDF report path

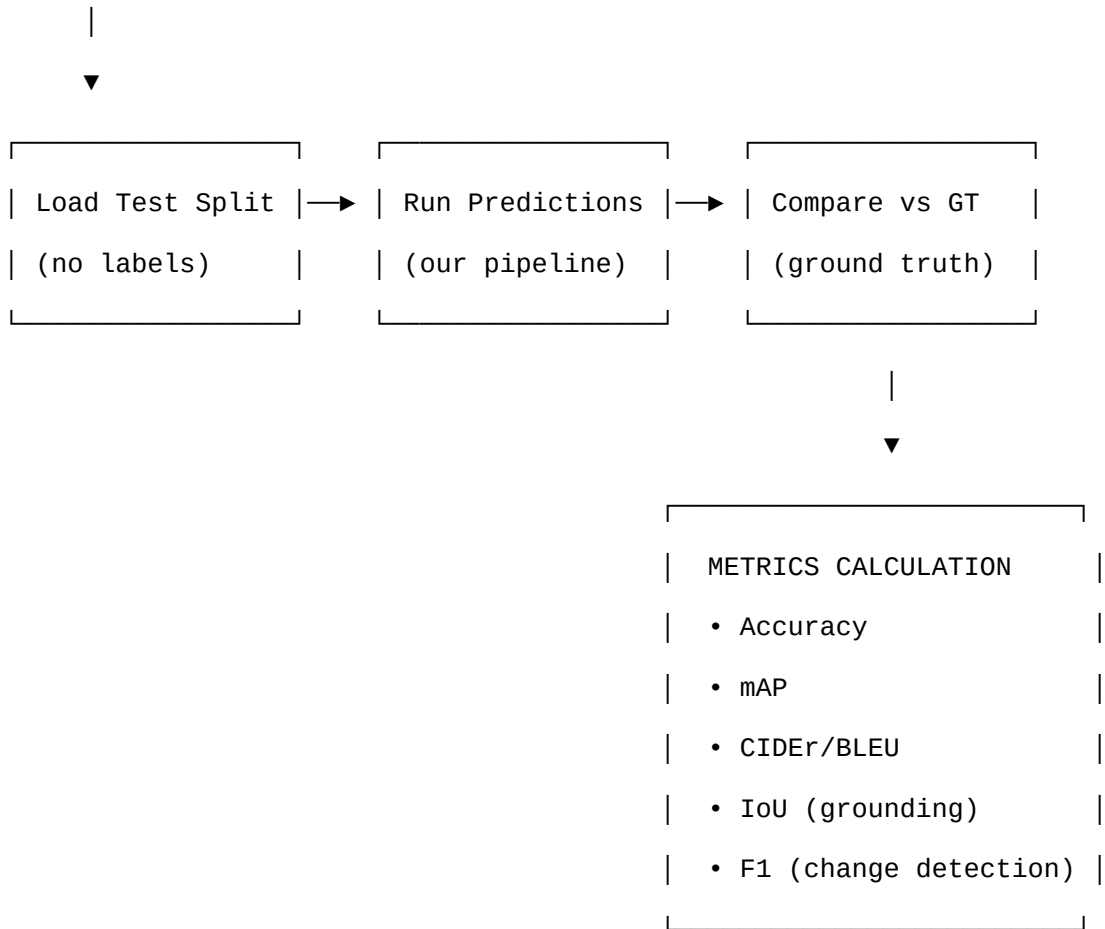
TABLE: execution_traces

id	UUID	Primary Key
task_id	UUID	FK → analysis_tasks
step_number	INTEGER	Step sequence
step_action	VARCHAR(100)	Action name
step_result	TEXT	Result/output of step
model_name	VARCHAR(100)	Model used in this step
parameters	JSONB	Parameters used
duration_sec	FLOAT	Step duration
timestamp	TIMESTAMP	Step timestamp

BENCHMARK	TASK	METRIC	OUR MODEL
RSVQA-HR	VQA	Accuracy (%)	RS-LLaVA fine-tuned
RSVQA-LR	VQA	Accuracy (%)	RS-LLaVA fine-tuned
VRSBench	Captioning	CIDEr, BLEU-4	GeoChat adapted
VRSBench	Grounding	mAP@0.5	Grounding-DINO-RS
VRSBench	VQA	Accuracy	RS-LLaVA
CDVQA	Change VQA	Accuracy (%)	ChangeFormer + Decoder
BigEarthNet	Classification	mAP	RemoteCLIP (adapted)
ISRO/SAC	All Tasks	Combined Score	Full Pipeline

EVALUATION FLOW:

Test Dataset



31 DEVELOPMENT TIMELINE
text

8-WEEK DEVELOPMENT ROADMAP

WEEK 1-2: FOUNDATION

	M1:	Project setup, Git, Docker skeleton, FastAPI + React scaffold
Day 1-2	M2:	GDAL/Rasterio GeoTIFF handling + metadata extraction
Day 3-4	M3:	MinIO + PostgreSQL + Redis setup
Day 5-6	M4:	RemoteCLIP integration (immediate RS-adapted encoder)
Day 7-8	M5:	Basic FastAPI upload + query endpoints
Day 9-10	M6:	React frontend scaffold + upload component
Day 11-14		

WEEK 3-4: CORE MODELS

	M7:	GeoChat VQA + Captioning integration
Day 15-17	M8:	Agentic controller skeleton (parser + classifier + selector)
Day 18-19	M9:	ChangeFormer integration + basic change map generation
Day 20-22	M10:	SAR preprocessing pipeline + dual encoder
Day 23-25	M11:	Grounding-DINO + SAM integration
Day 26-28		

WEEK 5-6: INTEGRATION + TRAINING

	M12:	BigEarthNet CLIP fine-tuning (start + run in background)
Day 29-31	M13:	Full agentic workflow integration + testing
Day 32-34		

M14: Confidence scoring system

Day 35-36

M15: Execution trace logging + UI panel

Day 37-38

M16: Frontend results panels + map visualization

Day 39-42

WEEK 7-8: POLISH + DEMO PREP

M17: PDF report generation

Day 43-44

M18: Benchmark evaluation on RSVQA, VRSBench, CDVQA

Day 45-47

M19: ISRO dataset format testing + optimization

Day 48-50

M20: Docker deployment + performance optimization

Day 51-52

M21: Demo preparation + bug fixing

Day 53-54

M22: Presentation + video demo creation

Day 55-56

COMPLETE WINNING CHECKLIST

text

JUDGE EVALUATION CHECKLIST

MANDATORY REQUIREMENTS:

- ☒

RS-adapted visual component (BigEarthNet + RemoteCLIP fine-tuning)

☒

Single-image VQA working on RSVQA benchmark

☒

Captioning OR grounding (implement BOTH for extra points)

☒

Bi-temporal change analysis (ChangeFormer + description)

☒

SAR-Optical cross-modal joint analysis

☒

Agentic orchestration with model registry

☒

GeoTIFF/TIFF format support via GDAL

BONUS WINNING FEATURES:

- ★ Confidence scores for all outputs
- ★ Visual change maps with color coding
- ★ Full execution audit trace in UI
- ★ PDF report download with all results
- ★ Before/after image comparison slider
- ★ Geographic coordinate output with changes
- ★ ISRO Cartosat-2S / RISAT compatibility
- ★ Fallback model if primary fails
- ★ Query history and session management
- ★ API documentation for integration